

1) Deadlock Debugging:-

Process P1:

lock(R1)

lock(R2)

Critical section

unlock(R2)

unlock(R1)

Process P2:

lock(R1)

lock(R2)

Critical section

unlock(R2)

unlock(R1)

9.4
⇒ By locking R1 first and R2 second in both Presence the circular wait is removed and the deadlock is prevented.

Condition causing Deadlock:

The deadlock occurs because:

- P1 locks R1 and waits for R2
- P2 locks R2 and waits for R1.

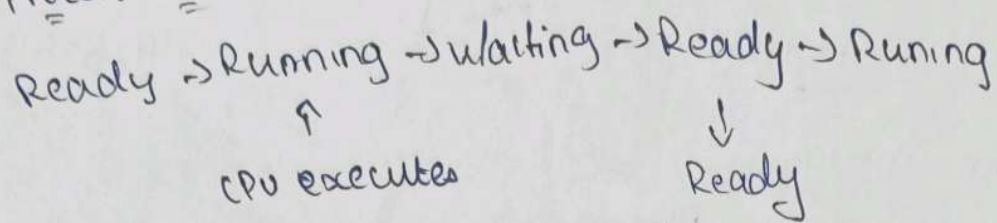
Optimisation to eliminate Deadlock Acquire the resource in the same order in both processes.

2) CPU utilisation Problem:

The repeated transitions show that Processes are spending most of their time in the waiting state

- Frequent I/O operation
- slow I/O devices causing long waiting time.
- Processes requesting I/O time immediately
- CPU remains idle while waiting for I/O

Process state diagram



Result:

- CPU utilisation increases
- CPU idle time decrease
- overall system throughput and performance improve

3) Memory Bottleneck Debugging:

- Code = 5 MB $\rightarrow$ Frequent Page Faults
- Heap = 150 MB $\rightarrow$ Slower Performance
- Stack = $25 \times 1 \text{ MB} = 25 \text{ MB}$
- Total = 180 MB (120 MB)

Fix optimizations:

1. Reduce Threads (Thread Pool)

- Limit threads to 8-10
- Less contention + fewer page faults

2. Optimize Heap Access:

- use contiguous data / object pooling
- Reduce fragmentation
- Improve locality

• many threads $\rightarrow$ ram

• heap data scattered $\rightarrow$ cache misses $\rightarrow$ page faults.

$\Rightarrow$ The issue is caused by excessive threading and inefficient heap access.

4) Starvation Debugging:

- Process P5 in Q3 - Batch, queued
- Never gets CPU time $\rightarrow$ starvation detected.

1) Q1 (High) $\rightarrow$ Interactive

2) Q2 $\rightarrow$ System (I/Os)

3) Q3 $\rightarrow$ (Lowest) $\rightarrow$ FCFS

Optimization:

1) Apply Aging Technique

2) Use Time Sharing Between Queues

2) The root cause of starvation is strictly scheduling in multilevel queues, where continuous execution of higher-priority processes prevents Q3 processes like P5 from running.

5) Context Switch overhead Debugging:-

Time Quantum = 1ms, burst time = 2ms $\rightarrow$ threads early

- very high of I/O wait $\rightarrow$ frequent block/unblock

Root cause

$\rightarrow$ Too small time quantum (more preemption)

$\rightarrow$ frequent I/O interrupts

Optimization

1) Slightly increase time quantum (2-4 ms)

2) Improve I/O handling (I/O-aware scheduling)

$\therefore$ context switching is high due to small quantum and frequent I/O waits

6) Producer-consumer Debugging

• Producer:

wait (empty)

wait (mutex)

insert (item)

signal (mutex)

signal (full)

Consumer

wait (full)

wait (mutex)

remove (item)

signal (mutex)

signal (empty)

⇒ The error is incorrect semaphore ordering,
locking mutex before checking empty/full blocks
other threads while waiting

⇒ Thread Performance Debugging

Continuous of the thread creation → CPU waste

Too many threads → context switching the devices
of the threads are not used reused → memory
keep increasing.

Fix:

we use the Thread Pool → reuse threads,
no creation overhead, Limit the active
threads, controls CPU & memory.

⇒ ∴ Problem is creating one thread per request

It slows due to high creation cost and fix
using the threads no creation overhead.

8) Banker's Algorithm Debugging:

1) Need matrix (Need = Max - Allocation)

$$P_1 (2-1, 1-0, 2-1) = (1, 1, 1)$$

$$P_2 (1-0, 2-1, 1-0) = (1, 1, 1)$$

$$P_3 (3-1, 2-1, 2-1) = (2, 1, 1)$$

2) Check request ($P_2 \rightarrow (1, 1, 1)$)

$\Rightarrow$ Available = (2, 1, 1), Request $\leq$ Available $\cdot$ request $\leq$ Need (A)

Safe : $P_2 \rightarrow P_1 \rightarrow P_3$

$\Rightarrow$ 1. Need matrix:

$$P_1 (1, 1, 1) \quad P_2 (1, 1, 1) \quad P_3 (2, 1, 1)$$

2. Request can be granted

3. System remains safe (no deadlock)

a) Deadlock Detection Debugging

1) Deadlock check

$$T_1 \rightarrow R_2 \rightarrow T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_4 \rightarrow T_4 \rightarrow R_1 \rightarrow T_1$$

Detection

• Deadlock detection checks for cycle in Resource Allocation Graph

cycle $\rightarrow$ all transactions waiting $\rightarrow$ no progress

Recovery Techniques

1) Abort one transaction (eg. T_4)

$\Rightarrow$ Breaks cycle quickly

2. Abort one transaction

→ minimum disruption & faster

3. Resource preemption

→ Take resource from one transaction

10) Monitor Debugging

Monitor Printer {

Boolean busy = false;

Procedure print()

{

while (busy)

wait();

busy = true;

busy = false;

Signal();

}

}

∴ Error is missing signal and wrong condition check

waiting process never make because no signal is sent.